

ACTIVITAT AVALUABLE AC1**Mòdul:** M0612- Desenvolupament web en entorn client**U1 – Fonaments de JavaScript****Professor:** Albert Guardiola**Data límit d'entrega:** 23/09/2026 23:59**Mètode d'entrega:** Per mitjà del Clickedu de l'assignatura.**Instruccions:** S'ha d'entregar un únic document amb el nom: **MP0616-U1-AC1-Nom_Alumne.js****Avaluació de la tasca:** La nota serà la del control posterior que es realitzarà a classe.**Resultats de l'aprenentatge:**

RA1, RA2, RA3, RA4

AC1 -Blindatge d'una aplicació JavaScript

Cas d'ús professional: Gestor de pressupostos de serveis

Rebeu una aplicació JavaScript ja desenvolupada per calcular pressupostos d'un servei professional.

L'aplicació permet:

- Introduir hores de servei.
- Introduir un preu/hora.
- Aplicar un descompte.
- Escollir un tipus de client.
- Calcular impostos.
- Aplicar una tarifa mínima.
- Mostrar el resultat.
- Consultar informació de compatibilitat i configuració.
- Treballar amb valors procedents del DOM.

El problema: l'aplicació funciona en alguns casos, però presenta comportaments incorrectes quan l'usuari introdueix determinats valors.

L'objectiu no és reescriure-la, sinó **analitzar el codi existent, detectar els problemes, consultar documentació i anar-lo blindant progressivament**, aplicant bones pràctiques d'enginyeria de programari i consultant recursos oficials del sector.

1. Context Professional i Filosofia del "Blindatge"

En entorns de desenvolupament professional (e-commerce, aplicacions financeres, panells de gestió *business2business*), la diferència entre un codi que "funciona en el cas ideal" i un codi **preparat per a producció (Enterprise Ready)** rau en la seva capacitat de resiliència. Un error sintàctic, una reassignació no controlada o una coerció implícita de tipus no només pot trencar la interfície d'usuari (UI), sinó que pot generar pèrdues econòmiques directes, bretxes de seguretat o problemes de compliment legal i fiscal.

Les aplicacions JavaScript modernes han de ser *defensives*: han de preveure entrades malformades de l'usuari, comportaments asíncrons inesperats, col·lisions en l'àrea d'execució i diferències entre entorns o navegadors. L'objectiu d'aquesta pràctica és agafar una aplicació amb múltiples fallades de disseny i sotmetre-la a un procés intensiu de diagnòstic, comprovació pràctica i refactorització per "blindar-la" completament.

2. Instruccions Generals de Treball

1. Analitza el codi de partida proporcionat al fitxer/annex adjunt.
2. Completa cada un dels exercicis proposats seguint la seqüència: **Diagnòstic** → **Comprovació Pràctica** → **Consulta de Recursos** → **Refactorització**.
3. Executa les proves pràctiques a la consola de desenvolupador del navegador (F12 / Ctrl+Shift+I) per verificar el comportament erroni abans d'aplicar la solució.
4. Utilitza exclusivament la documentació oficial (**MDN Web Docs**) i plataformes de compatibilitat (**Can I Use**) per fonamentar les teves respostes teòriques.

Bloc 1: Execució Segura, Declaració de Variables i Àmbits d'Execució

Temari cobert: Mode Estricte ('use strict'), variables globals accidentals, `let` vs `const` vs `var`, àmbit de bloc (*block scope*) vs àmbit de funció (*function scope*), i *hoisting*.

Exercici 1.1: Mode Estricte i Contaminació de l'Àmbit Global

- **Diagnòstic de l'error:** A la funció `getFormData()`, l'assignació de la variable `hours` es realitza sense cap paraula clau de declaració (`var`, `let` o `const`). En mode no estricte, JavaScript crea automàticament una variable global assignada a l'objecte global (`window` al navegador o `globalThis` en entorns moderns/Node.js).
- **Com ho pots comprovar? (Prova pràctica):**
 1. Carrega l'aplicació al navegador i obre la consola de desenvolupador (F12).
 2. Executa manualment des de la consola la funció `getFormData()`.
 3. Escribeu a la consola `window.hours` o `globalThis.hours`. Observa com retorna el valor de l'input en lloc de `undefined`. La variable s'ha convertit en una propietat global no desitjada.
- **Context professional:** En aplicacions complexes amb mòduls o llibreries de tercers (eines d'anàlisi, píxels de publicitat, xats de suport), les variables globals accidentals provoquen "bugs fantasma" per col·lisió de noms, on un script sobrescriu les dades de l'altre.
- **Tasca pràctica:**
 1. Insereix la directiva `'use strict'`; a la primera línia del fitxer JavaScript.
 2. Torna a executar `getFormData()` i verifica que la consola llança un `Uncaught ReferenceError: hours is not defined`.
 3. Cerca a **MDN Web Docs (Strict mode)** quines altres accions prohibeix el mode estricte.
 4. Declara la variable `hours` utilitzant *scoping* de bloc (`const` o `let`).

Exercici 1.2: Migració d'Àmbit d'Execució (`var` → `let` / `const`) i Hoisting

- **Diagnòstic de l'error:** L'ús de `var` atorga àmbit de funció i produeix *hoisting* (elevació de la declaració). A la funció `testTaxScope()`, la variable `tax` està declarada dins d'un bloc `if (true)`, però és accessible des de fora del bloc. A més, **constants** de configuració com `minimumPrice` o referències al DOM usen `var`, fent-les reassignables per accident.
- **Com ho pots comprovar? (Prova pràctica):**
 1. Executa la funció `testTaxScope()` des de la consola i comprova com imprimeix "Impost de prova: 21", demostrant que la variable ha escapat del bloc `if`.
 2. Escribeu a la consola `minimumPrice = 0;`. Observa com JavaScript ho permet sense cap avís, destruint la regla de negoci del preu mínim.
- **Tasca pràctica:**
 1. Substitueix totes les declaracions de referències immutables (elements HTML, arrays de configuració, valors límit) per `const`.
 2. Substitueix les variables que requereixin reassignació lògica per `let`.

3. Comprova que en canviar `var tax = 21` per `let tax = 21` a `testTaxScope()`, l'intent d'accedir a `tax` fora del bloc `if` llança correctament un `Uncaught ReferenceError`.

Bloc 2: Tipus de Dades, Parseig, Conversió i Seguretat de Valors

Temari cobert: Tipus de dades primitius, conversió explícita i implícita, `parseInt` vs `parseFloat` vs `Number()`, gestió de `NaN`, operador *Nullish Coalescing* (`??`) vs operador `OR` (`||`), i paràmetres per defecte en funcions.

Exercici 2.1: El Perill dels Valors Falsy i l'Operador ??

- **Diagnòstic de l'error:** A `calculateBudget()` es fa servir la condició `if (!discount)` { `discount = 10;` } després de fer un `parseInt`. Si l'usuari introdueix un descompte del 0% (escriu 0 a l'input), `parseInt("0")` retorna el número 0. Com que 0 és un valor *falsy*, !0 s'avalua com a `true` i el programa força el descompte a 10.
- **Com ho pots comprovar? (Prova pràctica):**
 1. Al formulari HTML, posa 10 hores, 100 €/hora, selecciona client `standard` i escriu manualment un 0 al camp de descompte.
 2. Fes clic a "Calcular". Observa com l'aplicació aplica un descompte de 100.00 € (10%) en lloc de 0.00 €.
- **Context professional:** Aplicar un descompte per defecte quan l'usuari ha indicat explícitament un descompte zero suposa una pèrdua directa de marge comercial en qualsevol plataforma de vendes.
- **Tasca pràctica:**
 1. Consulta a **MDN Web Docs** la llista dels 8 valors *falsy* en JavaScript (`false`, 0, -0, 0n, "", `null`, `undefined`, `NaN`).
 2. Analitza la diferència entre l'operador `||` (s'activa amb qualsevol valor *falsy*) i l'operador `??` (*Nullish Coalescing*, que NOMÉS s'activa amb `null` o `undefined`).
 3. Refactoritza la lògica per tal que un descompte de 0 sigui considerat un valor vàlid, i només s'assigni el valor per defecte si el camp està buit o no s'ha definit.

Exercici 2.2: Sanejament i Parseig Segur contra Contaminació NaN

- **Diagnòstic de l'error:** Si un usuari deixa un input buit o hi introdueix text, `parseFloat("")` o `parseFloat("abc")` retorna `NaN` (*Not a Number*). Aquest valor es propaga a totes les operacions matemàtiques posteriors (`NaN * 50 = NaN`), fent fallar les crides a `.toFixed(2)` o mostrant "NaN" a la interfície.
- **Com ho pots comprovar? (Prova pràctica):**
 1. Esborra el contingut de l'input d'hores (deixa'l buit) o escriu-hi text.
 2. Fes clic al botó de calcular i comprova com els camps de resultat de la UI mostren `NaN` o es produeix un error a la consola.
- **Tasca pràctica:**
 1. Dins de la funció `calculateBudget()`:
 - Converteix les entrades del formulari a número i aplica una validació amb `Number.isNaN()` i comprovació de valors negatius (`< 0`).
 - Implementa un tall d'execució defensiu fent servir `throw new Error("El teu missatge d'error")` quan les dades siguin invàlides.

- Verifica com el bloc try...catch de la funció processBudget() captura aquest error i el passa a renderError(error) per mostrar error.message a la interfície HTML, evitant que l'aplicació s'aturi o mostri NaN.
2. Refactoritza la funció calculateTax fent servir **paràmetres per defecte de l'ES6** per evitar valors undefined:

JavaScript

```
function calculateTax(clientType = "standard") { ... }
```

Bloc 3: Lògica Condicional, Precedència d'Operadors i Igualtat Estricta

Temari cobert: Igualtat estricta (===) vs no estricta (==), precedència d'operadors lògics (&& sobre ||), i simplificació d'estructures de control amb l'operador ternari (? :).

Exercici 3.1: Coerciò Implícita de Tipus i Igualtat Estricta

- **Diagnòstic de l'error:** A través de tot el codi s'utilitza l'operador d'igualtat feble == (ex: `clientType == "premium"`). Aquest operador realitza una coerció automàtica de tipus abans de comparar, provocant avaluacions inesperades.
- **Com ho pots comprovar? (Prova pràctica):**
 1. Obre la consola i avalua les següents expressions: `"0" == 0` (retorna `true`), `"" == false` (retorna `true`), `!null == undefined` (retorna `true`).
 2. Compara els resultats executant les mateixes expressions amb igualtat estricta (===): `"0" === 0` (retorna `false`).
- **Tasca pràctica:**
 1. Reemplaça totes les instàncies de `==` i `!=` per igualtat estricta `===` i desigualtat estricta `!==`.
 2. Refactoritza la funció `calculateTax` utilitzant l'operador condicional ternari (`? :`) per fer el codi més declaratiu.

Exercici 3.2: Corrupció de Condicionals per Precedència Lògica

- **Diagnòstic de l'error:** Es vol aplicar una reducció d'impostos als clients estàndard que sol·licitin un pressupost per més de 10h o més de 50 euros/hora (els clients Premium ja sempre tenen l'impost reduït). A `calculateBudget()` trobem la condició:

```
if (hours > 10 || price > 50 && formData.clientType == "standard") {  
  tax = 10; }  
}
```

A causa de les regles de precedència d'operadors, `&&` té major prioritat d'execució que `||`. La condició s'interpreta en realitat com: `(hours > 10) || (price > 50 && formData.clientType === "standard")`, de manera que la reducció d'impostos s'aplica per a tots els tipus de clients, erròniament.

El bug: Si un projecte té més de 10 hores (`hours > 10`), la primera part s'avalua com a `true` i l'operador `||` dona la condició per complerta immediatament. Això fa que el programa reassigni l'impost al 10% a qualsevol client no *standard* (com un client de tipus VIP o corporatiu), ignorant que la reducció d'IVA per volum d'hores estava pensada únicament per als clients de tipus *standard*.

- **Com ho pots comprovar? (Prova pràctica):**
 1. Configura el formulari HTML amb aquestes dades exactes:
 - **Hores:** 12 (és > 10)
 - **Preu/Hora:** 20 €
 - **Tipus de client:** VIP (o qualsevol tipus no `premium` / no `standard`)
 2. Executa el càlcul amb el codi actual.

3. Comprova la diferència entre el resultat obtingut i el valor esperat segons la lògica de negoci:

Concepte	Codi amb Bug	Valor Esperat (Lògica Corregida)
Subtotal	240.00 € (12h × 20€)	240.00 € (12h × 20€)
Descompte (10%)	24.00 €	24.00 €
Subtotal amb descompte	216.00 €	216.00 €
Taxa d'impost aplicada	10% (Bug: entra a l'if per tenir >10h)	21% (Correcte: no és client standard)
Import Impost	21.60 €	45.36 €
TOTAL FINAL	237.60 €	261.36 €

• **Context professional:** Els errors de precedència d'operadors són extremadament perillosos perquè no generen un error de sintaxi a la consola; el programa s'executa correctament però calcula valors erronis. En sistemes de facturació, això pot comportar liquidacions d'IVA incorrectes davant l'administració tributària.

• **Tasca pràctica:**

1. Consulta la taula de **Precedència d'operadors a MDN Web Docs (Operator precedence)** per verificar la prioritat d'execució de `&&` respecte a `||`.
2. Reescriu la condició utilitzant parèntesis explícits per agrupar la lògica correctament: la condició d'hores (`hours > 10`) o preu (`price > 50`) s'ha d'avaluar en conjunt I A MÉS el client ha de ser obligatòriament `standard`:

```
const isLargeProject = hours > 10 || price > 50;
if (isLargeProject && formData.clientType === "standard") {
  tax = 10;
}
```

3. Torna a introduir les dades de prova (12 hores, 20 €/h, client `VIP`) i comprova que el total final ara és exactament **261.36 €**.

Bloc 4: Iteració Moderna, Estructures de Dades i Compatibilitat

Temari cobert: Bucle `for` clàssic vs `for...of` vs mètodes d'arrays (`reduce`), i verificació de compatibilitat de característiques de l'idioma amb eines del sector.

Exercici 4.1: Modernització de Bucles i Disseny Declaratiu

- **Diagnòstic de l'error:** Les funcions `listServices()` i `calculateServicesTotal()` utilitzen bucles `for` clàssics basats en comptadors manuals. Aquest patró és propens a errors de límit (*off-by-one errors*).
- **Com ho pots comprovar? (Prova pràctica):**
 1. Modifica temporalment la condició de sortida del bucle a `calculateServicesTotal()` per: `index <= services.length`.
 2. Executa la funció a la consola i observa com el resultat dona `NaN` en intentar sumar un element inexistent (`undefined`).
- **Tasca pràctica:**
 1. Refactoritza `listServices()` per utilitzar el bucle modern `for...of`.
 2. Refactoritza `calculateServicesTotal()` per utilitzar `for...of` o el mètode `Array.prototype.reduce()`.

Exercici 4.2: Anàlisi de Compatibilitat de Navegadors (Browser Compatibility)

- **Diagnòstic de l'error:** Característiques modernes com `globalThis` o l'operador `??` (introduïts a ECMAScript 2020) poden fallar si l'aplicació s'executa en navegadors antics o entorns *legacy*.
- **Com ho pots comprovar? (Prova pràctica):**
 1. Accedeix a **Can I Use** (caniuse.com).
 2. Cerca el nivell de suport per a `Nullish coalescing operator` i `globalThis`.
 3. Comprova quines versions de navegadors quedarien excloses si s'executa el codi directament sense cap procés de construcció.
- **Tasca pràctica:**
 1. Documenta breument quines eines (transpiladors com Babel, o *polyfills*) s'utilitzen en entorns industrials per garantir la compatibilitat amb navegadors antics.